

Java Multithreading - Cheat Sheet

Multithreading lets one program do several things at once. The JVM maps Java threads to OS threads (or to virtual threads in Java 21+), and the OS scheduler decides who runs when. That sharing is what makes concurrent code fast and also what makes it dangerous when shared state isn't managed carefully.

Thread Basics

A `Thread` is the basic unit of execution. Every Java program starts with at least one thread (the `main` thread). You create more threads to run work in parallel or to avoid blocking the main flow.

```
Thread t = new Thread(() -> System.out.println("hello from another thread"));
t.start();
```

`start()` schedules the thread on the JVM. `run()` calls the code directly on the current thread, which defeats the whole point.

Creating Threads

Three common ways. Pick based on what you need.

1. Implement Runnable

The classic approach. Cleaner because your class can extend something else.

```
public class PrintTask implements Runnable {
    @Override
    public void run() {
        System.out.println("running");
    }
}

Thread t = new Thread(new PrintTask());
t.start();
```

2. Lambda (Runnable shortcut)

Java 8+. Same Runnable, way less code.

```
Thread t = new Thread(() -> System.out.println("running"));
t.start();
```

3. Extend Thread

Works, but couples your class to `Thread` and blocks single inheritance.

```
public class PrintThread extends Thread {  
    @Override  
    public void run() {  
        System.out.println("running");  
    }  
}  
  
new PrintThread().start();
```

For most code, prefer `Runnable`. Use `Callable` when the task returns a value.

```
Callable<Integer> task = () -> 42;
```

Thread Lifecycle

State	What it means
NEW	Created but <code>start()</code> not called yet
RUNNABLE	Eligible to run, may or may not be on CPU right now
BLOCKED	Waiting for a monitor lock
WAITING	Waiting indefinitely (e.g., <code>wait()</code> , <code>join()</code>)
TIMED_WAITING	Waiting with a timeout (e.g., <code>sleep(ms)</code> , <code>wait(ms)</code>)
TERMINATED	Done running

```
Thread t = new Thread(() -> doWork());  
System.out.println(t.getState()); // NEW  
t.start();  
System.out.println(t.getState()); // RUNNABLE
```

Key Thread Methods

Method	What it does
<code>start()</code>	Schedules the thread to run
<code>run()</code>	The body of work, don't call directly
<code>join()</code>	Waits for the thread to finish
<code>sleep(ms)</code>	Pauses the current thread for ms milliseconds
<code>interrupt()</code>	Signals the thread to stop (cooperative)
<code>isInterrupted()</code>	Checks the interrupt flag
<code>currentThread()</code>	Returns the running thread
<code>yield()</code>	Hints to the scheduler to let others run (rarely useful)

synchronized

The most common locking primitive in Java. Two forms.

Synchronized method locks on the instance (or the class, for static methods).

```
public class Counter {  
    private int count = 0;  
  
    public synchronized void increment() {  
        count++;  
    }  
  
    public synchronized int get() {  
        return count;  
    }  
}
```

Synchronized block locks on any object you pick. Finer control.

```
private final Object lock = new Object();  
  
public void update() {  
    synchronized (lock) {  
        // critical section  
    }  
}
```

Rules:

- Only one thread holds the lock at a time.
 - Other threads block until the lock is released.
 - Locks are reentrant. The same thread can re-acquire its own lock.
 - Lock on a private object dedicated to locking. Not on `this` or the class object, since external code might lock on them too.
-

volatile

Marks a field so reads and writes go straight to main memory, skipping per-thread caches. Also provides ordering guarantees (happens-before).

```
public class StopFlag {  
    private volatile boolean running = true;  
  
    public void stop() {  
        running = false;  
    }  
  
    public void loop() {  
        while (running) {  
            // do work  
        }  
    }  
}
```

Use `volatile` when:

- One thread writes, multiple read (a flag).
- You need to publish a reference safely.

Don't use it when:

- You need atomicity for compound operations. `count++` is still not atomic even with `volatile`.
-

wait / notify / notifyAll

The original primitives for thread coordination. Must be called inside a `synchronized` block on the same object.

```

synchronized (lock) {
    while (!ready) {
        lock.wait();
    }
    // proceed
}

// Another thread
synchronized (lock) {
    ready = true;
    lock.notifyAll();
}

```

Rules:

- Always call `wait()` inside a `while` loop, not an `if`. Spurious wakeups exist.
- `notify()` wakes one waiter. `notifyAll()` wakes everyone. Prefer `notifyAll()` unless you know exactly which one to wake.
- These are low-level. For new code, prefer `java.util.concurrent` primitives like `BlockingQueue`, `CountDownLatch`, or `CyclicBarrier`.

ExecutorService and Thread Pools

Skip raw `Thread` objects in production code. Use an `ExecutorService` to manage a pool of reusable threads.

```

ExecutorService pool = Executors.newFixedThreadPool(4);

pool.submit(() -> doWork());
pool.submit(() -> doMoreWork());

pool.shutdown();
pool.awaitTermination(1, TimeUnit.MINUTES);

```

Common pool factories:

Factory	Behavior
<code>newFixedThreadPool(n)</code>	Fixed n threads, unbounded task queue
<code>newCachedThreadPool()</code>	Grows and shrinks as needed, idle threads die after 60s
<code>newSingleThreadExecutor()</code>	One thread, tasks run sequentially
<code>newScheduledThreadPool(n)</code>	Run tasks after a delay or periodically
<code>newWorkStealingPool()</code>	ForkJoinPool, good for many small tasks
<code>newVirtualThreadPerTaskExecutor()</code>	Virtual threads (Java 21+)

Always call `shutdown()` (or use try-with-resources in Java 19+). Otherwise the JVM won't exit cleanly.

Future and CompletableFuture

Submit a task and get a handle to its result.

```
ExecutorService pool = Executors.newFixedThreadPool(4);

Future<Integer> future = pool.submit(() -> {
    Thread.sleep(1000);
    return 42;
});

Integer result = future.get(); // blocks until done
```

`CompletableFuture` is the modern, richer version. Supports chaining and async composition.

```
CompletableFuture
    .supplyAsync(() -> fetchUser(id))
    .thenApply(User::getEmail)
    .thenAccept(email -> System.out.println(email))
    .exceptionally(ex -> {
        log.error("failed", ex);
        return null;
    });
```

Key methods:

- `supplyAsync(Supplier)` : start an async computation that returns a value.
- `runAsync(Runnable)` : start an async computation with no return.

- `thenApply(Function)` : transform the result.
 - `thenAccept(Consumer)` : consume the result with no return.
 - `thenCompose(Function)` : chain another async call (flat-map style).
 - `thenCombine(other, BiFunction)` : combine results from two futures.
 - `exceptionally(Function)` : handle errors and recover.
 - `allOf(...)` / `anyOf(...)` : wait for all or any of multiple futures.
-

Locks (java.util.concurrent.locks)

More flexible than `synchronized` .

```
private final ReentrantLock lock = new ReentrantLock();

public void update() {
    lock.lock();
    try {
        // critical section
    } finally {
        lock.unlock();
    }
}
```

Why pick a `Lock` over `synchronized` :

- `tryLock()` : attempt to acquire without blocking.
- `tryLock(timeout, unit)` : give up after a timeout.
- `lockInterruptibly()` : respond to interrupts while waiting.
- Fairness: `new ReentrantLock(true)` favors the longest-waiting thread.

`ReadWriteLock` for read-heavy workloads:

```

private final ReadWriteLock rwLock = new ReentrantReadWriteLock();

public String read() {
    rwLock.readLock().lock();
    try {
        return data;
    } finally {
        rwLock.readLock().unlock();
    }
}

public void write(String value) {
    rwLock.writeLock().lock();
    try {
        data = value;
    } finally {
        rwLock.writeLock().unlock();
    }
}

```

Multiple readers can hold the read lock at once. Writers get exclusive access.

Atomic Classes

Lock-free thread-safe operations on single values. Lives in `java.util.concurrent.atomic`.

```

AtomicInteger counter = new AtomicInteger(0);

counter.incrementAndGet();    // ++counter
counter.getAndIncrement();    // counter++
counter.addAndGet(5);
counter.compareAndSet(10, 20); // CAS: set to 20 only if currently 10

AtomicReference<User> currentUser = new AtomicReference<>(null);
currentUser.compareAndSet(null, new User("alice"));

```

Use when one variable is updated by multiple threads. Faster than a `synchronized` counter for simple cases.

Common types: `AtomicInteger`, `AtomicLong`, `AtomicBoolean`, `AtomicReference<T>`, `LongAdder` (faster than `AtomicLong` under high contention).

Concurrent Collections

Class	Use case
<code>ConcurrentHashMap</code>	General thread-safe map, default choice
<code>CopyOnWriteArrayList</code>	Read-heavy, rare writes
<code>ConcurrentLinkedQueue</code>	Unbounded thread-safe FIFO queue
<code>LinkedBlockingQueue</code> / <code>ArrayBlockingQueue</code>	Producer-consumer queues with blocking
<code>ConcurrentSkipListMap</code>	Sorted concurrent map

```
ConcurrentHashMap<String, Integer> map = new ConcurrentHashMap<>();
map.computeIfAbsent("key", k -> expensiveComputation());
map.merge("count", 1, Integer::sum);
```

For most thread-safe map needs, `ConcurrentHashMap` is the answer.

Common Problems

Race Condition

Two threads read-modify-write a shared variable without synchronization. The final value depends on timing.

```
int count = 0;

// Two threads run this
void increment() {
    count++; // not atomic: read, add, write
}
```

Fix: use `synchronized`, `AtomicInteger`, or a `Lock`.

Deadlock

Two threads each hold a lock the other one needs.

```
// Thread 1
synchronized (lockA) {
    synchronized (lockB) {
        // ...
    }
}

// Thread 2
synchronized (lockB) {
    synchronized (lockA) {
        // ...
    }
}
```

Both threads wait forever. Fix: always acquire locks in the same order across threads. Or use `tryLock` with a timeout.

Livelock

Threads aren't blocked, but they keep changing state in response to each other and make no progress. Like two people sidestepping in a hallway.

Starvation

A thread can't get CPU time or a lock because higher-priority or greedier threads keep grabbing them.

Virtual Threads (Java 21+)

Lightweight threads managed by the JVM, not the OS. Millions of them can run on a handful of carrier threads. Designed for blocking I/O workloads.

```
// Single virtual thread
Thread.startVirtualThread(() -> doWork());

// Virtual thread executor
try (ExecutorService pool = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 10_000; i++) {
        pool.submit(() -> handleRequest());
    }
}
```

Use virtual threads for I/O-heavy work (HTTP servers, DB clients). Skip them for CPU-bound work or where you need to bound parallelism explicitly.

Best Practices

- **Reach for high-level primitives.** `ExecutorService` , `CompletableFuture` , concurrent collections beat raw threads and `wait / notify` .
 - **Make shared state immutable when you can.** No mutation, no race conditions.
 - **Lock on private objects.** Not on `this` or class literals. External code might lock on them too.
 - **Keep critical sections short.** Long-held locks block other threads.
 - **Always release Lock objects in a finally block.** Otherwise a thrown exception leaves the lock held forever.
 - **Always shut down ExecutorService** . Otherwise the JVM hangs at exit.
 - **Don't catch InterruptedException and ignore it.** Either propagate it, or restore the flag with `Thread.currentThread().interrupt()` .
-

Common Gotchas

- `count++` is three operations (read, add, write). Not atomic. Use `AtomicInteger` or `synchronized` .
 - `volatile` doesn't make compound operations atomic. It fixes visibility and ordering, that's it.
 - `HashMap` is not thread-safe. Concurrent modification can corrupt internal structures and cause infinite loops on `get()` . Use `ConcurrentHashMap` .
 - `Thread.sleep()` doesn't release locks. The thread holds them while sleeping.
 - `notify()` wakes one waiter but doesn't say which. If different waiters expect different conditions, you can wake the wrong one. Use `notifyAll()` or `Condition` objects.
 - `Thread.stop()` , `Thread.suspend()` , and `Thread.resume()` are deprecated and unsafe. Use interrupt-based cancellation.
 - Daemon threads (`setDaemon(true)`) don't keep the JVM alive. Use them for background work that should die with the program.
 - Virtual threads get pinned to their carrier thread inside a `synchronized` block or a native call. For blocking sections, prefer `ReentrantLock` to avoid pinning.
-

Quick Reference

Concept	Use it for
Thread	Direct thread creation, rare in modern code
Runnable / Callable	A task you want a thread to run
synchronized	Simple mutual exclusion
volatile	Single-variable visibility and ordering
wait / notify	Low-level coordination (prefer concurrent utilities)
ExecutorService	Managed thread pool
Future	Handle to an async result
CompletableFuture	Composable async pipelines
ReentrantLock	Flexible locking with try and timeout
ReadWriteLock	Read-heavy workloads
Atomic*	Lock-free single-value updates
ConcurrentHashMap	Thread-safe map, the default
BlockingQueue	Producer-consumer patterns
Virtual threads	I/O-heavy workloads (Java 21+)